

IHK Trainer

Multiple Choice Prüfungssimulation

100 Fragen mit Lösungen

Quelle: Syntax Institut GitHub

Frage 1 Networking

Was beschreibt der Begriff „Client“ im Web-Kontext am besten?

Ein Computer, der ausschließlich Datenbanken hostet

✓ **Ein Programm (z.B. Browser), das Anfragen an einen Server stellt**

Ein Gerät, das nur WLAN-Signale verstärkt

Ein Server, der HTTP-Requests ignoriert

Antwort: Ein Programm (z.B. Browser), das Anfragen an einen Server stellt

Frage 2 Networking

Welche Aussagen über IP-Adressen sind korrekt?

✓ **IPv4 besteht typischerweise aus vier Zahlenblöcken (z.B. 192.168.0.1)**

✓ **127.0.0.1 ist die Loopback-Adresse (localhost)**

Eine IP-Adresse ist immer ein frei wählbarer Benutzername

IPv4-Adressen bestehen aus 8 Zahlenblöcken

Antworten:

- IPv4 besteht typischerweise aus vier Zahlenblöcken (z.B. 192.168.0.1)
- 127.0.0.1 ist die Loopback-Adresse (localhost)

Frage 3 Networking

Wofür wird ein DNS-Server hauptsächlich verwendet?

✓ **Um Domainnamen in IP-Adressen aufzulösen**

Um HTML in CSS zu konvertieren

Um JavaScript im Browser schneller auszuführen

Um Passwörter zu verschlüsseln

Antwort: Um Domainnamen in IP-Adressen aufzulösen

Frage 4 Web Basics

Welche Teile können zu einer vollständigen URL gehören?

✓ **Protokoll (Scheme), Host/Domain, Pfad (Path), Query-Parameter**

Nur Domain und Dateiname, mehr ist nicht erlaubt

Protokoll, IP-Adresse, nur ein Query-Parameter

✓ **Protokoll, Host/Domain, Pfad, Fragment (Hash)**

Antworten:

- Protokoll (Scheme), Host/Domain, Pfad (Path), Query-Parameter
- Protokoll, Host/Domain, Pfad, Fragment (Hash)

Frage 5 Web Basics

Was ist der wichtigste Unterschied zwischen HTTP und HTTPS?

✓ **HTTPS verschlüsselt die Verbindung (typisch via TLS), HTTP nicht**

HTTP ist immer schneller als HTTPS

HTTPS funktioniert nur in mobilen Apps

HTTP kann keine Bilder übertragen

Antwort: HTTPS verschlüsselt die Verbindung (typisch via TLS), HTTP nicht

Frage 6 HTTP

Welche HTTP-Methoden werden typischerweise für CRUD verwendet?

✓ **GET (Read), POST (Create), PUT/PATCH (Update), DELETE (Delete)**

READ, CREATE, UPDATE, REMOVE

FETCH, PUSH, CHANGE, DROP

OPEN, WRITE, EDIT, CLOSE

Antwort: GET (Read), POST (Create), PUT/PATCH (Update), DELETE (Delete)

Frage 7 HTTP

Welche HTTP-Statuscodes bedeuten typischerweise Erfolg?

✓ **2xx (z.B. 200, 201)**

3xx (z.B. 301, 302)

4xx (z.B. 404, 401)

5xx (z.B. 500, 503)

Antwort: 2xx (z.B. 200, 201)

Frage 8 HTTP

Welche Aussagen zu HTTP-Requests sind korrekt?

✓ **Header enthalten Metadaten wie Content-Type oder Authorization**

Der Body ist nur bei GET erlaubt

✓ **Query-Parameter stehen typischerweise nach einem ? in der URL**

Cookies werden nie im Kontext von HTTP genutzt

Antworten:

- Header enthalten Metadaten wie Content-Type oder Authorization
- Query-Parameter stehen typischerweise nach einem ? in der URL

Frage 9 HTML

Welche HTML-Elemente sind semantisch sinnvoll für die grobe Seitenstruktur?

✓ **<header>, <main>, <footer>**

<div>, ,

<table>, <tr>, <td>

<script>, <style>, <meta>

Antwort: <header>, <main>, <footer>

Frage 10 HTML

Wofür wird das alt-Attribut bei verwendet?

Es definiert, wie stark ein Bild komprimiert wird

✓ **Es beschreibt den Bildinhalt als Text (wichtig für Accessibility/Fallback)**

Es ersetzt automatisch die src-URL

Es sorgt dafür, dass Bilder schneller laden

Antwort: Es beschreibt den Bildinhalt als Text (wichtig für Accessibility/Fallback)

Frage 11 HTML

Welche Aussagen über Formulare in HTML sind korrekt?

- ✓ **Das required-Attribut erzwingt, dass ein Feld ausgefüllt wird (Browser-Validierung)**
- ✓ **<label> verbessert Usability/Accessibility, wenn es einem Input zugeordnet ist**

Formulare können keine Daten an einen Server senden

Ein `<button type="submit">` löst nie eine Formularaktion aus

Antworten:

- Das required-Attribut erzwingt, dass ein Feld ausgefüllt wird (Browser-Validierung)
- `<label>` verbessert Usability/Accessibility, wenn es einem Input zugeordnet ist

Frage 12 HTML

Welche Elemente werden typischerweise für Tabellen benötigt?

- ✓ **`<table>`, `<tr>`, `<td>`, `<th>`**

`<grid>`, `<row>`, `<cell>`

`<tablelist>`, `<tu>`, `<item>`

`<tab>`, `<line>`, `<col>`

Antwort: `<table>`, `<tr>`, `<td>`, `<th>`

Frage 13 CSS

Was bewirkt box-sizing: border-box;?

- ✓ **Padding und Border werden in width/height eingerechnet**

Alle Borders werden entfernt

Das Element wird automatisch zentriert

Nur der Content zählt zur Breite, Padding wird addiert

Antwort: Padding und Border werden in width/height eingerechnet

Frage 14 CSS

Welche Aussagen zum Box-Modell sind korrekt?

- ✓ **Margin ist der Außenabstand eines Elements**
- ✓ **Padding ist der Abstand zwischen Content und Border**
- Border ist immer transparent und unsichtbar
- ✓ **Content ist der innere Bereich (z.B. Text, Bild)**

Antworten:

- Margin ist der Außenabstand eines Elements
- Padding ist der Abstand zwischen Content und Border
- Content ist der innere Bereich (z.B. Text, Bild)

Frage 15 CSS

Was ist der Unterschied zwischen display: none und visibility: hidden?

- ✓ **visibility: hidden lässt den Platz im Layout bestehen, display: none nicht**

display: none versteckt nur Text, nicht Bilder

visibility: hidden entfernt das Element komplett aus dem DOM

Es gibt keinen Unterschied

Antwort: visibility: hidden lässt den Platz im Layout bestehen, display: none nicht

Frage 16 CSS

Welche Aussagen über Selektoren sind korrekt?

- ✓ **Selektoren bestimmen, auf welche Elemente Styles angewendet werden**
- ✓ **.klasse selektiert Elemente mit class="klasse"**
- ✓ **#id selektiert Elemente mit id="id"**

Selektoren funktionieren nur mit Inline-Styles

Antworten:

- Selektoren bestimmen, auf welche Elemente Styles angewendet werden
- .klasse selektiert Elemente mit class="klasse"
- #id selektiert Elemente mit id="id"

Frage 17 CSS

Welche Eigenschaften helfen bei responsiven Bildern?

✓ **max-width: 100%; height: auto;**

width: fixed; height: fixed;
responsive: true;
size: fluid;

Antwort: max-width: 100%; height: auto;

Frage 18 CSS Layout

Wofür eignet sich Flexbox typischerweise besonders gut?

✓ **Eindimensionale Layouts (Reihe ODER Spalte) und Ausrichtung**

Zweidimensionale Raster über die gesamte Seite (Rows und Columns gleichzeitig)

Nur für Print-Stylesheets

Nur für Tabellen

Antwort: Eindimensionale Layouts (Reihe ODER Spalte) und Ausrichtung

Frage 19 CSS Layout

Wofür eignet sich CSS Grid typischerweise besonders gut?

✓ **Komplexe zweidimensionale Layouts (Rows UND Columns)**

Nur für kleine Buttons und Icons

Ausschließlich für Animationen

Nur für Inline-Elemente

Antwort: Komplexe zweidimensionale Layouts (Rows UND Columns)

Frage 20 CSS

Was sind Media Queries?

✓ **Bedingungen in CSS, um Styles abhängig von Geräteeigenschaften anzuwenden**

Ein JavaScript-Feature zum Abfragen von APIs

Ein HTML-Tag für Responsive Design

Eine Datenbankabfrage

Antwort: Bedingungen in CSS, um Styles abhängig von Geräteeigenschaften anzuwenden

Frage 21 CSS

Welche Aussagen über Breakpoints sind korrekt?

- ✓ **Breakpoints sind Stellen, an denen sich das Layout für unterschiedliche Bildschirmgrößen anpasst**
Breakpoints sind ausschließlich in HTML definiert
- ✓ **Breakpoints werden oft in Media Queries verwendet**
Breakpoints sind HTTP-Fehlercodes

Antworten:

- Breakpoints sind Stellen, an denen sich das Layout für unterschiedliche Bildschirmgrößen anpasst
- Breakpoints werden oft in Media Queries verwendet

Frage 22 JavaScript

Welche Werte sind in JavaScript typischerweise „falsy“?

- ✓ **0**
- ✓ **null**
- ✓ **"" (leerer String)**
- [] (leeres Array)

Antworten:

- 0
- null
- "" (leerer String)

Frage 23 JavaScript

Was ist der Unterschied zwischen null und undefined?

- ✓ **null ist oft ein bewusst gesetzter leerer Wert, undefined ist „nicht gesetzt“/fehlt**
undefined ist ein Zahlentyp, null ist ein String
Beide sind immer identisch und austauschbar
- ✓ **undefined wird oft automatisch vergeben, null setzt man typischerweise explizit**

Antworten:

- null ist oft ein bewusst gesetzter leerer Wert, undefined ist „nicht gesetzt“/fehlt
- undefined wird oft automatisch vergeben, null setzt man typischerweise explizit

Frage 24 JavaScript

Was macht der Nullish Coalescing Operator (??)?

- ✓ **Er gibt den rechten Wert zurück, nur wenn links null oder undefined ist**
Er ist identisch zu || in jedem Fall
- ✓ **Er behandelt 0 und "" nicht automatisch als „leer“ (anders als ||)**
Er vergleicht zwei Werte wie ===

Antworten:

- Er gibt den rechten Wert zurück, nur wenn links null oder undefined ist
- Er behandelt 0 und "" nicht automatisch als „leer“ (anders als ||)

Frage 25 JavaScript

Welche Aussagen zu let und const sind korrekt?

- ✓ **let kann neu zugewiesen werden, const nicht**
const verhindert jede Änderung am Inhalt von Objekten/Arrays
- ✓ **const muss bei Deklaration initialisiert werden**
let ist nur in Funktionen gültig (function scope)

Antworten:

- let kann neu zugewiesen werden, const nicht
- const muss bei Deklaration initialisiert werden

Frage 26 JavaScript

Wie erstellt man ein leeres Array korrekt?

- ✓ **const a = [];**
const a = {};
- ✓ **const a = new Array();**
const a = {};

Antworten:

- const a = [];
- const a = new Array();

Frage 27 JavaScript

Was ist ein Array in JavaScript?

- ✓ **Eine geordnete Liste von Werten**
Eine einzelne Zahl
Eine ungeordnete Menge ohne Reihenfolge
Ein spezieller HTML-Tag

Antwort: Eine geordnete Liste von Werten

Frage 28 JavaScript

Womit beginnt die Zählung bei Array-Indizes in JavaScript?

- Bei 1
- ✓ **Bei 0**
- Bei -1
- Das ist zufällig und variiert je Browser

Antwort: Bei 0

Frage 29 JavaScript

Was macht die Array-Methode map() typischerweise?

- Sie verändert immer das Originalarray und gibt nichts zurück
- ✓ **Sie erzeugt ein neues Array**
- ✓ **Sie wendet eine Funktion auf jedes Element an**
- ✓ **Die Länge des neuen Arrays entspricht normalerweise der alten**

Antworten:

- Sie erzeugt ein neues Array
- Sie wendet eine Funktion auf jedes Element an
- Die Länge des neuen Arrays entspricht normalerweise der alten

Frage 30 DOM

Wofür steht DOM und was ist es?

- ✓ **Document Object Model: die Struktur, die der Browser aus HTML erzeugt**
Data Object Mode: ein JavaScript-Modus für Datenbanken
Design Operation Method: eine CSS-Layouttechnik
Document Order Map: eine Liste von CSS-Regeln

Antwort: Document Object Model: die Struktur, die der Browser aus HTML erzeugt

Frage 31 DOM

Was macht `document.getElementById()`?

Es erstellt ein neues Element mit einer ID

✓ **Es findet ein Element anhand seiner eindeutigen ID**

Es gibt immer eine Liste aller Elemente zurück

Es löscht ein Element aus dem DOM

Antwort: Es findet ein Element anhand seiner eindeutigen ID

Frage 32 DOM

Welche Aufgaben übernimmt `classList` typischerweise?

✓ **CSS-Klassen hinzufügen/entfernen/togglen**

HTTP-Header bearbeiten

Elemente in der Datenbank speichern

Die Reihenfolge von Arrays sortieren

Antwort: CSS-Klassen hinzufügen/entfernen/togglen

Frage 33 JavaScript

Was ist der ternäre Operator (`? :`) ?

✓ **Eine Kurzform für einfache if-else-Ausdrücke**

Eine Methode zum Erstellen von Arrays

Ein Operator, der drei Zahlen addiert

Ein React-spezifisches Feature

Antwort: Eine Kurzform für einfache if-else-Ausdrücke

Frage 34 JavaScript

Welche Aussage beschreibt JSON am besten?

✓ **Ein Datenformat (Text), das oft für API-Austausch genutzt wird**

Ein binäres Bildformat

Ein CSS-Preprocessor

Ein JavaScript-Framework

Antwort: Ein Datenformat (Text), das oft für API-Austausch genutzt wird

Frage 35 JavaScript

Welche Funktionen gehören zu JSON in JavaScript?

✓ **`JSON.parse()` wandelt einen JSON-String in ein Objekt um**

✓ **`JSON.stringify()` wandelt ein Objekt in einen JSON-String um**

`JSON.keys()` gibt alle Schlüssel eines Objekts zurück

`JSON.toArray()` wandelt JSON direkt in ein Array um

Antworten:

• `JSON.parse()` wandelt einen JSON-String in ein Objekt um

• `JSON.stringify()` wandelt ein Objekt in einen JSON-String um

Frage 36 APIs

Was ist eine API?

✓ **Eine Schnittstelle, über die Programme Daten austauschen/aufrufen können**

Ein HTML-Tag für Formulare

Eine Datenbanktabelle

Ein Browser-Plugin für CSS

Antwort: Eine Schnittstelle, über die Programme Daten austauschen/aufrufen können

Frage 37 Async JS

Welche Aussagen über async/await sind korrekt?

- ✓ **await kann innerhalb einer async-Funktion verwendet werden**
- ✓ **await macht asynchronen Code lesbarer, indem es wie „synchron“ wirkt**
await blockiert immer den gesamten Browser-Thread vollständig
- ✓ **async-Funktionen geben typischerweise ein Promise zurück**

Antworten:

- await kann innerhalb einer async-Funktion verwendet werden
- await macht asynchronen Code lesbarer, indem es wie „synchron“ wirkt
- async-Funktionen geben typischerweise ein Promise zurück

Frage 38 Async JS

Wie fängt man Fehler bei einer fetch-Anfrage robust ab?

- ✓ **Mit try/catch bei async/await oder .catch() bei Promises**

Nur mit if/else, try/catch funktioniert nicht

Mit console.log(), das ersetzt Fehlerbehandlung

Gar nicht, fetch wirft keine Fehler

Antwort: Mit try/catch bei async/await oder .catch() bei Promises

Frage 39 Tooling

Wofür werden Developer Tools im Browser typischerweise genutzt?

- ✓ **HTML/CSS inspizieren und live anpassen**
- ✓ **Netzwerk-Requests (Network Tab) analysieren**
- ✓ **JavaScript debuggen (Breakpoints, Console)**

Server-RAM physisch erweitern

Antworten:

- HTML/CSS inspizieren und live anpassen
- Netzwerk-Requests (Network Tab) analysieren
- JavaScript debuggen (Breakpoints, Console)

Frage 40 Accessibility

Welche Maßnahmen verbessern typischerweise Accessibility (Barrierefreiheit)?

- ✓ **Sinnvolle semantische HTML-Tags (z.B. <nav>, <main>)**
- ✓ **Alt-Texte für Bilder, wenn sie Inhalt transportieren**
Nur Farben als einziges Mittel zur Informationsvermittlung
- ✓ **Labels für Form-Inputs**

Antworten:

- Sinnvolle semantische HTML-Tags (z.B. <nav>, <main>)
- Alt-Texte für Bilder, wenn sie Inhalt transportieren
- Labels für Form-Inputs

Frage 41 Security

Welche Aussagen zu XSS (Cross-Site Scripting) sind korrekt?

- ✓ **XSS bedeutet, dass unerwünschter Script-Code in Seiten eingeschleust werden kann**
XSS hat nur mit Netzwerkhardware zu tun
- ✓ **Escaping/Sanitizing von User-Input hilft, XSS zu vermeiden**
XSS ist identisch mit CSS Grid

Antworten:

- XSS bedeutet, dass unerwünschter Script-Code in Seiten eingeschleust werden kann
- Escaping/Sanitizing von User-Input hilft, XSS zu vermeiden

Frage 42 React Basics

Was ist React am treffendsten?

- ✓ **Eine JavaScript-Bibliothek zum Bauen von Benutzeroberflächen**
Eine Programmiersprache
Ein Backend-Server
Ein CSS-Framework

Antwort: Eine JavaScript-Bibliothek zum Bauen von Benutzeroberflächen

Frage 43 React Basics

Warum teilt man React-Apps in Komponenten auf?

- ✓ **Code wird übersichtlicher und besser wartbar**
- ✓ **Komponenten können wiederverwendet werden**
Komponenten ersetzen automatisch jede API
- ✓ **UI und Logik lassen sich sinnvoll kapseln**

Antworten:

- Code wird übersichtlicher und besser wartbar
- Komponenten können wiederverwendet werden
- UI und Logik lassen sich sinnvoll kapseln

Frage 44 React

Was sind Props in React?

- ✓ **Daten/Eigenschaften, die von außen an eine Komponente übergeben werden**
Ein Ersatz für CSS
Ein interner Zustand, der nur der Komponente gehört
Ein Browser-Feature für HTTP

Antwort: Daten/Eigenschaften, die von außen an eine Komponente übergeben werden

Frage 45 React

Welche Aussagen zu State in React sind korrekt?

- ✓ **State ist der aktuelle Zustand einer Komponente, der sich ändern kann**
- ✓ **Eine State-Änderung kann ein Re-Render auslösen**
State sollte immer direkt mutiert werden (state.value = ...)
- ✓ **useState liefert typischerweise [wert, setter]**

Antworten:

- State ist der aktuelle Zustand einer Komponente, der sich ändern kann
- Eine State-Änderung kann ein Re-Render auslösen
- useState liefert typischerweise [wert, setter]

Frage 46 React Hooks

Welche Regeln gelten für React Hooks?

- ✓ **Hooks nur in React-Komponenten oder anderen Hooks verwenden**
Hooks dürfen in Schleifen und Bedingungen beliebig aufgerufen werden

✓ **Hooks sollten immer in der gleichen Reihenfolge aufgerufen werden**

Hooks funktionieren nur in Klassenkomponenten

Antworten:

- Hooks nur in React-Komponenten oder anderen Hooks verwenden
- Hooks sollten immer in der gleichen Reihenfolge aufgerufen werden

Frage 47 React Hooks

Wofür wird useEffect typischerweise verwendet?

✓ **Für Nebenwirkungen wie Daten laden, Subscriptions, Timer**

Um JSX zu ersetzen

Um Props zu definieren

✓ **Um auf Änderungen bestimmter Werte zu reagieren (Dependency Array)**

Antworten:

- Für Nebenwirkungen wie Daten laden, Subscriptions, Timer
- Um auf Änderungen bestimmter Werte zu reagieren (Dependency Array)

Frage 48 React Forms

Was ist ein kontrolliertes Input-Feld (controlled component)?

✓ **Ein Input, dessen Wert aus React-State kommt und via onChange aktualisiert wird**

Ein Input, der ohne value-Prop besser funktioniert

Ein Input, der nur per document.getElementById() gelesen wird

Ein Input, der immer disabled ist

Antwort: Ein Input, dessen Wert aus React-State kommt und via onChange aktualisiert wird

Frage 49 React Router

Warum nutzt man React Router in Single Page Applications (SPAs)?

✓ **Um URL-basierte Navigation ohne komplettes Neuladen zu ermöglichen**

Um Datenbanktabellen im Browser zu erstellen

Um CSS automatisch zu generieren

✓ **Um verschiedene „Seiten“/Views über Routen zu organisieren**

Antworten:

- Um URL-basierte Navigation ohne komplettes Neuladen zu ermöglichen
- Um verschiedene „Seiten“/Views über Routen zu organisieren

Frage 50 React Performance

Welche Aussagen über unnötige Re-Renders und Optimierung sind korrekt?

✓ **Unnötige Re-Renders können Performance verschlechtern**

✓ **React.memo kann helfen, Re-Renders bei gleichen Props zu vermeiden**

Man sollte immer sofort alles optimieren, bevor es funktioniert

✓ **Optimierung ist oft nach funktionaler Korrektheit und bei echten Problemen sinnvoll**

Antworten:

- Unnötige Re-Renders können Performance verschlechtern
- React.memo kann helfen, Re-Renders bei gleichen Props zu vermeiden
- Optimierung ist oft nach funktionaler Korrektheit und bei echten Problemen sinnvoll

Frage 51 HTML

Welche Aussagen über semantisches HTML sind korrekt?

- ✓ **Semantische Tags verbessern die Struktur und Verständlichkeit des Dokuments**
- ✓ **Semantische Tags können Accessibility (z.B. Screenreader) unterstützen**
Semantisches HTML ersetzt CSS komplett
- ✓ **Semantisches HTML kann SEO positiv beeinflussen**

Antworten:

- Semantische Tags verbessern die Struktur und Verständlichkeit des Dokuments
- Semantische Tags können Accessibility (z.B. Screenreader) unterstützen
- Semantisches HTML kann SEO positiv beeinflussen

Frage 52 HTML

Welche Tags sind typischerweise für Links und Navigation zuständig?

- ✓ **<a>**
- ✓ **<nav>**
- <link>
- <section>

Antworten:

- <a>
- <nav>

Frage 53 HTML

Welche Aussagen über das <script>-Tag sind korrekt?

- ✓ **Mit defer wird das Script nach dem Parsen von HTML ausgeführt**
- ✓ **Mit async wird das Script geladen und ausgeführt, sobald es verfügbar ist**
<script> kann nur im <head> stehen
- ✓ **JavaScript kann über <script> in eine Seite eingebunden werden**

Antworten:

- Mit defer wird das Script nach dem Parsen von HTML ausgeführt
- Mit async wird das Script geladen und ausgeführt, sobald es verfügbar ist
- JavaScript kann über <script> in eine Seite eingebunden werden

Frage 54 HTML

Welche Aussagen über das <link>-Element sind korrekt?

- ✓ **Es wird häufig genutzt, um externe CSS-Dateien einzubinden**
Es muss immer im <body> stehen
- ✓ **rel="stylesheet" kennzeichnet eine Stylesheet-Datei**
- ✓ **href enthält typischerweise den Pfad zur Ressource**

Antworten:

- Es wird häufig genutzt, um externe CSS-Dateien einzubinden
- rel="stylesheet" kennzeichnet eine Stylesheet-Datei
- href enthält typischerweise den Pfad zur Ressource

Frage 55 CSS

Welche Aussagen über CSS-Spezifität (specificity) sind korrekt?

- ✓ **ID-Selektoren sind in der Regel spezifischer als Klassen-Selektoren**
- ✓ **Inline-Styles sind oft sehr spezifisch**
Je mehr Selektoren, desto geringer immer die Spezifität
!important sollte man immer standardmäßig nutzen

Antworten:

- ID-Selektoren sind in der Regel spezifischer als Klassen-Selektoren
- Inline-Styles sind oft sehr spezifisch

Frage 56 CSS

Welche Einheiten sind in CSS relativ (relativ zur Schriftgröße oder zum Viewport)?

- ✓ **em**
- ✓ **rem**
- px
- ✓ **vw**

Antworten:

- em
- rem
- vw

Frage 57 CSS

Welche Aussagen zu position: absolute sind korrekt?

- ✓ **Das Element wird aus dem normalen Dokumentenfluss genommen**
- ✓ **Die Position bezieht sich auf den nächsten positionierten Vorfahren (nicht static)**
Es verhält sich immer wie position: fixed
- ✓ **Ohne positionierten Vorfahren kann es sich auf das Dokument/Viewport beziehen (je nach Kontext)**

Antworten:

- Das Element wird aus dem normalen Dokumentenfluss genommen
- Die Position bezieht sich auf den nächsten positionierten Vorfahren (nicht static)
- Ohne positionierten Vorfahren kann es sich auf das Dokument/Viewport beziehen (je nach Kontext)

Frage 58 CSS

Welche Aussagen zu z-index sind korrekt?

- ✓ **z-index wirkt nur bei positionierten Elementen (nicht static) oder in bestimmten Layout-Kontexten**
- ✓ **Ein höherer z-index-Wert bedeutet meist „weiter vorne“**
z-index steuert die Schriftgröße
z-index funktioniert nur in Tabellen

Antworten:

- z-index wirkt nur bei positionierten Elementen (nicht static) oder in bestimmten Layout-Kontexten
- Ein höherer z-index-Wert bedeutet meist „weiter vorne“

Frage 59 CSS

Welche Aussagen über Flexbox-Ausrichtung sind korrekt?

- ✓ **justify-content richtet entlang der Hauptachse aus**
- ✓ **align-items richtet entlang der Querachse aus**
- ✓ **flex-direction bestimmt, ob die Hauptachse Reihe oder Spalte ist**
gap ist in Flexbox nicht erlaubt

Antworten:

- justify-content richtet entlang der Hauptachse aus
- align-items richtet entlang der Querachse aus
- flex-direction bestimmt, ob die Hauptachse Reihe oder Spalte ist

Frage 60 CSS

Welche Aussagen über CSS Grid sind korrekt?

- ✓ **grid-template-columns definiert die Spalten**
- ✓ **grid-template-rows definiert die Zeilen**
- ✓ **Grid eignet sich gut für zweidimensionale Layouts**
Grid kann keine Abstände zwischen Zellen definieren

Antworten:

- grid-template-columns definiert die Spalten
- grid-template-rows definiert die Zeilen
- Grid eignet sich gut für zweidimensionale Layouts

Frage 61 CSS

Welche Aussagen über Transitionen in CSS sind korrekt?

- ✓ **transition-duration** legt fest, wie lange die Animation dauert
- ✓ **transition-property** legt fest, welche Eigenschaft animiert wird
- Transitionen können nur mit JavaScript ausgelöst werden
- ✓ **Transitionen eignen sich für einfache Zustandswechsel (z.B. :hover)**

Antworten:

- transition-duration legt fest, wie lange die Animation dauert
- transition-property legt fest, welche Eigenschaft animiert wird
- Transitionen eignen sich für einfache Zustandswechsel (z.B. :hover)

Frage 62 CSS

Welche Aussagen über CSS-Keyframes sind korrekt?

- ✓ **@keyframes** definiert Zwischenzustände einer Animation
- ✓ **animation-name** verweist auf eine @keyframes-Definition
- Keyframes sind nur für SVG möglich
- ✓ **animation-iteration-count** kann festlegen, wie oft die Animation wiederholt wird

Antworten:

- @keyframes definiert Zwischenzustände einer Animation
- animation-name verweist auf eine @keyframes-Definition
- animation-iteration-count kann festlegen, wie oft die Animation wiederholt wird

Frage 63 JavaScript

Welche Aussagen über === (strikte Gleichheit) sind korrekt?

- ✓ **=== vergleicht Wert und Typ**
- ✓ **=== führt keine Typumwandlung wie == durch**
- === ist nur für Zahlen erlaubt
- ✓ **=== ist in JavaScript der Standardvergleich für exakte Vergleiche**

Antworten:

- === vergleicht Wert und Typ
- === führt keine Typumwandlung wie == durch
- === ist in JavaScript der Standardvergleich für exakte Vergleiche

Frage 64 JavaScript

Welche Aussagen über Funktionen in JavaScript sind korrekt?

- ✓ **Funktionen können Parameter entgegennehmen**
- ✓ **Funktionen können Werte mit return zurückgeben**
- Eine Funktion kann nur einmal im Programm aufgerufen werden
- ✓ **Funktionen sind wiederverwendbare Code-Blöcke**

Antworten:

- Funktionen können Parameter entgegennehmen
- Funktionen können Werte mit return zurückgeben
- Funktionen sind wiederverwendbare Code-Blöcke

Frage 65 JavaScript

Welche Aussagen über Arrow Functions sind korrekt?

- ✓ **Arrow Functions nutzen die Syntax () => {}**
- Arrow Functions sind immer benannt und können nicht anonym sein
- ✓ **Arrow Functions können eine kürzere Schreibweise für Funktionen sein**
- ✓ **Arrow Functions haben ein anderes this-Verhalten als function() in vielen Fällen**

Antworten:

- Arrow Functions nutzen die Syntax () => {}
- Arrow Functions können eine kürzere Schreibweise für Funktionen sein

- Arrow Functions haben ein anderes this-Verhalten als function() in vielen Fällen

Frage 66 JavaScript

Welche Aussagen über Scope sind korrekt?

- ✓ **let und const sind block-scoped**
- ✓ **var ist function-scoped**
- ✓ **Scope beschreibt, wo Variablen sichtbar/verwendbar sind**
Scope ist ein CSS-Konzept

Antworten:

- let und const sind block-scoped
- var ist function-scoped
- Scope beschreibt, wo Variablen sichtbar/verwendbar sind

Frage 67 JavaScript

Welche Aussagen über Hoisting sind korrekt?

- ✓ **Funktionsdeklarationen werden oft vor ihrer Position nutzbar (hoisted)**
let/const werden wie var behandelt und sind immer ohne Fehler vor Deklaration nutzbar
- ✓ **var-Deklarationen werden „hochgezogen“, aber initialisiert erst später**
Hoisting bedeutet, dass der Code schneller ausgeführt wird

Antworten:

- Funktionsdeklarationen werden oft vor ihrer Position nutzbar (hoisted)
- var-Deklarationen werden „hochgezogen“, aber initialisiert erst später

Frage 68 JavaScript

Welche Aussagen über Object.keys() sind korrekt?

- ✓ **Es liefert ein Array der eigenen (enumerable) Schlüsselnamen eines Objekts**
Es liefert ein Array aller Werte eines Objekts
Es funktioniert nur für Arrays, nicht für Objekte
- ✓ **Es ist nützlich, um über Properties zu iterieren**

Antworten:

- Es liefert ein Array der eigenen (enumerable) Schlüsselnamen eines Objekts
- Es ist nützlich, um über Properties zu iterieren

Frage 69 JavaScript

Welche Aussagen zu Optional Chaining (?.) sind korrekt?

- ✓ **Es verhindert Fehler beim Zugriff auf tief verschachtelte Properties, wenn etwas undefined/null ist**
Es ersetzt immer try/catch
- ✓ **Es kann auch bei Funktionsaufrufen genutzt werden (z.B. obj.fn?.())**
Es ist identisch zu && in jedem Fall

Antworten:

- Es verhindert Fehler beim Zugriff auf tief verschachtelte Properties, wenn etwas undefined/null ist
- Es kann auch bei Funktionsaufrufen genutzt werden (z.B. obj.fn?.())

Frage 70 JavaScript

Welche Aussagen zu Event Listenern sind korrekt?

- ✓ **addEventListener("click", ...) reagiert auf Klicks**
- ✓ **Events sind z.B. Klicks, Tastatur-Eingaben oder Submit**
Events entstehen nur im Backend
- ✓ **Man kann mit preventDefault() Standardverhalten verhindern (z.B. bei Formularen)**

Antworten:

- addEventListener("click", ...) reagiert auf Klicks
- Events sind z.B. Klicks, Tastatur-Eingaben oder Submit

- Man kann mit `preventDefault()` Standardverhalten verhindern (z.B. bei Formularen)

Frage 71 Web APIs

Welche Aussagen über localStorage sind korrekt?

- ✓ **Daten bleiben in der Regel nach einem Reload bestehen**
localStorage speichert nur für die aktuelle Browser-Session und wird danach gelöscht
- ✓ **Es speichert Strings (Objekte müssen oft per JSON.stringify gespeichert werden)**
- ✓ **localStorage ist pro Origin (Domain/Protokoll/Port) getrennt**

Antworten:

- Daten bleiben in der Regel nach einem Reload bestehen
- Es speichert Strings (Objekte müssen oft per JSON.stringify gespeichert werden)
- localStorage ist pro Origin (Domain/Protokoll/Port) getrennt

Frage 72 Web APIs

Welche Aussagen über sessionStorage sind korrekt?

- ✓ **Daten sind typischerweise nur für den Tab/Session gültig**
Daten bleiben für immer erhalten, auch nach Browser-Neustart
- ✓ **Es speichert Strings ähnlich wie localStorage**
- ✓ **Es ist pro Origin getrennt**

Antworten:

- Daten sind typischerweise nur für den Tab/Session gültig
- Es speichert Strings ähnlich wie localStorage
- Es ist pro Origin getrennt

Frage 73 HTTP

Welche Aussagen über Cookies sind korrekt?

- ✓ **Cookies können automatisch bei Requests an die Domain gesendet werden (je nach Einstellungen)**
Cookies werden nur in JavaScript-Dateien gespeichert
- ✓ **HttpOnly-Cookies sind typischerweise nicht per JavaScript auslesbar**
Cookies werden nie für Authentifizierung verwendet

Antworten:

- Cookies können automatisch bei Requests an die Domain gesendet werden (je nach Einstellungen)
- HttpOnly-Cookies sind typischerweise nicht per JavaScript auslesbar

Frage 74 HTTP

Welche Aussagen zu CORS sind korrekt?

- ✓ **CORS regelt, ob eine Website Ressourcen von einer anderen Origin abrufen darf**
- ✓ **CORS ist ein Browser-Sicherheitsmechanismus (nicht „Server blockiert“, sondern Browser setzt Regeln um)**
CORS bedeutet, dass HTTPS nicht benötigt wird
CORS hat nichts mit APIs zu tun

Antworten:

- CORS regelt, ob eine Website Ressourcen von einer anderen Origin abrufen darf
- CORS ist ein Browser-Sicherheitsmechanismus (nicht „Server blockiert“, sondern Browser setzt Regeln um)

Frage 75 Performance

Welche Maßnahmen verbessern oft Web-Performance?

- ✓ **Bilder optimieren (Größe/Format) und lazy loading nutzen**
- ✓ **Unnötiges JavaScript reduzieren (Bundle kleiner halten)**
Alles in eine riesige CSS-Datei ohne Struktur packen
- ✓ **Caching nutzen (Browser/Server), wo sinnvoll**

Antworten:

- Bilder optimieren (Größe/Format) und lazy loading nutzen
- Unnötiges JavaScript reduzieren (Bundle kleiner halten)
- Caching nutzen (Browser/Server), wo sinnvoll

Frage 76 Performance

Welche Aussagen über Rendering sind korrekt?

- ✓ **Der Browser parst HTML und baut daraus das DOM**
- ✓ **CSS beeinflusst Layout und Paint**
- ✓ **JavaScript kann das DOM verändern und dadurch Reflows/Repaints auslösen**

Rendering passiert nur beim ersten Seitenaufruf, nie wieder

Antworten:

- Der Browser parst HTML und baut daraus das DOM
- CSS beeinflusst Layout und Paint
- JavaScript kann das DOM verändern und dadurch Reflows/Repaints auslösen

Frage 77 Tooling

Welche Aussagen über npm sind korrekt?

- ✓ **npm verwaltet JavaScript-Pakete/Dependencies**
- ✓ **package.json beschreibt u.a. Abhängigkeiten und Scripts**
npm kann nur für React genutzt werden
- ✓ **npm install installiert Abhängigkeiten (typischerweise in node_modules)**

Antworten:

- npm verwaltet JavaScript-Pakete/Dependencies
- package.json beschreibt u.a. Abhängigkeiten und Scripts
- npm install installiert Abhängigkeiten (typischerweise in node_modules)

Frage 78 Tooling

Was macht npm run build typischerweise in einem Frontend-Projekt?

- ✓ **Es erstellt eine Produktionsversion (optimiert) der App**
Es startet immer automatisch den Dev-Server
- ✓ **Es erzeugt oft einen Output-Ordner (z.B. dist oder build)**
Es löscht alle Abhängigkeiten

Antworten:

- Es erstellt eine Produktionsversion (optimiert) der App
- Es erzeugt oft einen Output-Ordner (z.B. dist oder build)

Frage 79 Deployment

Was bedeutet Deployment im Web-Kontext typischerweise?

Eine App lokal entwickeln

- ✓ **Eine App im Internet/auf einem Server bereitstellen**
- ✓ **Eine App für Nutzer zugänglich machen**

Den Code nur als PDF exportieren

Antworten:

- Eine App im Internet/auf einem Server bereitstellen
- Eine App für Nutzer zugänglich machen

Frage 80 React

Welche Aussagen über JSX sind korrekt?

- ✓ **JSX ist eine Syntax-Erweiterung, um UI ähnlich wie HTML zu beschreiben**
- ✓ **In JSX kann man JavaScript-Ausdrücke mit {} einbetten**
JSX wird vom Browser direkt ohne Build-Tool verstanden
JSX muss immer in Anführungszeichen geschrieben werden

Antworten:

- JSX ist eine Syntax-Erweiterung, um UI ähnlich wie HTML zu beschreiben
- In JSX kann man JavaScript-Ausdrücke mit {} einbetten

Frage 81 React

Was ist beim Rendern von Listen in React wichtig?

- ✓ **Man nutzt häufig `map()`, um aus Daten JSX-Elemente zu erzeugen**
- ✓ **Jedes Listenelement sollte einen stabilen key erhalten**
 - key sollte idealerweise bei jedem Render zufällig neu generiert werden
 - React braucht keys nie

Antworten:

- Man nutzt häufig `map()`, um aus Daten JSX-Elemente zu erzeugen
- Jedes Listenelement sollte einen stabilen key erhalten

Frage 82 React

Welche Aussagen über keys in React sind korrekt?

- ✓ **Keys helfen React, Elemente zwischen Renders wiederzuerkennen**
- ✓ **Stabile Keys verbessern oft Performance und verhindern UI-Bugs bei Listen**
 - Index als key ist immer perfekt und ohne Nachteile
- ✓ **Keys müssen nur innerhalb einer Liste eindeutig sein**

Antworten:

- Keys helfen React, Elemente zwischen Renders wiederzuerkennen
- Stabile Keys verbessern oft Performance und verhindern UI-Bugs bei Listen
- Keys müssen nur innerhalb einer Liste eindeutig sein

Frage 83 React

Welche Aussagen über Conditional Rendering in JSX sind korrekt?

- ✓ **Mit `&&` kann man etwas nur rendern, wenn links truthy ist**
- ✓ **Mit dem ternären Operator kann man zwischen zwei UI-Varianten wählen**
 - if-Statements können direkt in JSX (ohne Hilfsfunktion) stehen
- ✓ **Man kann auch früh returnen (early return), um Bedingungen abzubilden**

Antworten:

- Mit `&&` kann man etwas nur rendern, wenn links truthy ist
- Mit dem ternären Operator kann man zwischen zwei UI-Varianten wählen
- Man kann auch früh returnen (early return), um Bedingungen abzubilden

Frage 84 React Hooks

Welche Aussagen über `useState` sind korrekt?

- ✓ **`useState` gibt einen aktuellen Wert und eine Setter-Funktion zurück**
- ✓ **Der Setter triggert typischerweise einen Re-Render**
 - Man sollte State immer direkt mutieren (z.B. `state.count++`)
- ✓ **`useState` kann Initialwerte als Funktion erhalten (lazy init)**

Antworten:

- `useState` gibt einen aktuellen Wert und eine Setter-Funktion zurück
- Der Setter triggert typischerweise einen Re-Render
- `useState` kann Initialwerte als Funktion erhalten (lazy init)

Frage 85 React Hooks

Welche Aussagen über das Dependency Array in `useEffect` sind korrekt?

- ✓ **Es steuert, wann der Effekt erneut ausgeführt wird**
- ✓ **Ein leeres Array bedeutet: Effekt läuft nur einmal nach dem ersten Render (typisch)**
- ✓ **Ohne Array läuft der Effekt nach jedem Render**
 - Das Array speichert automatisch State-Werte dauerhaft

Antworten:

- Es steuert, wann der Effekt erneut ausgeführt wird
- Ein leeres Array bedeutet: Effekt läuft nur einmal nach dem ersten Render (typisch)
- Ohne Array läuft der Effekt nach jedem Render

Frage 86 React Hooks

Welche Aussagen über useRef sind korrekt?

- ✓ **useRef kann eine mutable Referenz speichern, ohne Re-Renders auszulösen**
- ✓ **useRef eignet sich u.a. für DOM-Zugriff (z.B. Input fokussieren)**
 - useRef ersetzt immer useState
 - useRef-Werte werden bei jedem Render automatisch zurückgesetzt

Antworten:

- useRef kann eine mutable Referenz speichern, ohne Re-Renders auszulösen
- useRef eignet sich u.a. für DOM-Zugriff (z.B. Input fokussieren)

Frage 87 React

Was bedeutet „State anheben“ (lifting state up)?

- ✓ **State in eine übergeordnete Komponente verschieben, damit mehrere Kinder ihn nutzen können**
 - State in eine tiefere Komponente verschieben, um ihn zu verstecken
 - State komplett löschen und durch Props ersetzen
 - State automatisch global machen

Antwort: State in eine übergeordnete Komponente verschieben, damit mehrere Kinder ihn nutzen können

Frage 88 React

Welche Aussagen über children-Props sind korrekt?

- ✓ **children enthält Inhalte, die zwischen den Component-Tags stehen**
- ✓ **children wird genutzt, um Komponenten „komponierbar“ zu machen**
 - children kann nur Text sein, niemals andere Komponenten
- ✓ **children ist eine Standard-Prop in React**

Antworten:

- children enthält Inhalte, die zwischen den Component-Tags stehen
- children wird genutzt, um Komponenten „komponierbar“ zu machen
- children ist eine Standard-Prop in React

Frage 89 React

Welche Aussagen über React Context sind korrekt?

- ✓ **Context hilft, Daten „globaler“ verfügbar zu machen, ohne Props durch viele Ebenen zu reichen**
 - Context ist ausschließlich für CSS gedacht
- ✓ **Man greift häufig über useContext() auf Context zu**
 - Context ersetzt jede Form von State-Management immer vollständig

Antworten:

- Context hilft, Daten „globaler“ verfügbar zu machen, ohne Props durch viele Ebenen zu reichen
- Man greift häufig über useContext() auf Context zu

Frage 90 React

Welche Aussagen über fetch in React sind korrekt?

- ✓ **fetch wird genutzt, um Daten vom Server zu laden**
 - fetch muss immer in render() aufgerufen werden
- ✓ **API-Aufrufe macht man oft in useEffect, um Nebenwirkungen zu kapseln**
- ✓ **fetch liefert typischerweise ein Promise zurück**

Antworten:

- fetch wird genutzt, um Daten vom Server zu laden
- API-Aufrufe macht man oft in useEffect, um Nebenwirkungen zu kapseln
- fetch liefert typischerweise ein Promise zurück

Frage 91 React Router

Welche Aussagen über Client-Side Routing sind korrekt?

- ✓ **Navigation kann ohne vollständigen Seitenreload passieren**
- ✓ **Die URL kann sich ändern, obwohl nur eine HTML-Seite geladen wurde**
Routing bedeutet, dass der Server immer jede Seite als separate HTML-Datei ausliefert
- ✓ **SPAs nutzen oft Client-Side Routing**

Antworten:

- Navigation kann ohne vollständigen Seitenreload passieren
- Die URL kann sich ändern, obwohl nur eine HTML-Seite geladen wurde
- SPAs nutzen oft Client-Side Routing

Frage 92 React Router

Wozu dient eine Catch-All Route (z.B. /*) typischerweise?

- ✓ **Sie fängt unbekannte Pfade ab**
- ✓ **Sie zeigt oft eine NotFound/404-Komponente**
- ✓ **Sie matcht häufig erst, wenn keine andere Route passt**
Sie hat immer automatisch Vorrang vor allen anderen Routen

Antworten:

- Sie fängt unbekannte Pfade ab
- Sie zeigt oft eine NotFound/404-Komponente
- Sie matcht häufig erst, wenn keine andere Route passt

Frage 93 React Performance

Welche Aussagen über React.memo sind korrekt?

- ✓ **React.memo kann Re-Renders vermeiden, wenn Props unverändert sind**
React.memo funktioniert nur bei Klassenkomponenten
- ✓ **React.memo ist eine Optimierung und nicht immer nötig**
React.memo ersetzt useEffect vollständig

Antworten:

- React.memo kann Re-Renders vermeiden, wenn Props unverändert sind
- React.memo ist eine Optimierung und nicht immer nötig

Frage 94 React Performance

Warum können anonyme Funktionen in JSX manchmal problematisch sein?

- ✓ **Sie können bei jedem Render eine neue Referenz erzeugen**
- ✓ **Das kann unnötige Re-Renders in Child-Komponenten verursachen**
Sie sind grundsätzlich verboten in React
- ✓ **Sie machen Code manchmal schwerer lesbar**

Antworten:

- Sie können bei jedem Render eine neue Referenz erzeugen
- Das kann unnötige Re-Renders in Child-Komponenten verursachen
- Sie machen Code manchmal schwerer lesbar

Frage 95 Testing

Welche Aussagen über Unit-Tests sind korrekt?

- ✓ **Unit-Tests testen eine kleine Einheit (z.B. Funktion/Komponente) isoliert**
Unit-Tests ersetzen alle manuellen Tests vollständig
- ✓ **Unit-Tests helfen, Fehler früh zu erkennen**
Unit-Tests testen immer das komplette System-Ende-zu-Ende

Antworten:

- Unit-Tests testen eine kleine Einheit (z.B. Funktion/Komponente) isoliert
- Unit-Tests helfen, Fehler früh zu erkennen

Frage 96 Testing

Welche Aussagen über das Ziel von Testing sind korrekt?

✓ **Fehler früh erkennen und Regressionen vermeiden**

Code automatisch schöner machen ohne Review

✓ **Vertrauen in Änderungen erhöhen**

APIs vollständig ersetzen

Antworten:

- Fehler früh erkennen und Regressionen vermeiden
- Vertrauen in Änderungen erhöhen

Frage 97 Architecture

Welche Aussagen über „Single Source of Truth“ sind korrekt?

✓ **Es bedeutet, dass Daten/Zustand eine eindeutige, verlässliche Quelle haben sollen**

Es bedeutet, dass jede Komponente ihren eigenen Zustand immer duplizieren soll

✓ **Es kann helfen, Inkonsistenzen zu vermeiden**

Es bedeutet, dass alles nur im Backend liegen darf

Antworten:

- Es bedeutet, dass Daten/Zustand eine eindeutige, verlässliche Quelle haben sollen
- Es kann helfen, Inkonsistenzen zu vermeiden

Frage 98 Architecture

Warum ist eine saubere Ordnerstruktur in Projekten oft wichtig?

✓ **Bessere Wartbarkeit und Orientierung**

✓ **Bessere Teamarbeit (Konventionen, Auffindbarkeit)**

Automatisch schnelleres Rendering im Browser

✓ **Reduziert Merge-Konflikte häufig indirekt**

Antworten:

- Bessere Wartbarkeit und Orientierung
- Bessere Teamarbeit (Konventionen, Auffindbarkeit)
- Reduziert Merge-Konflikte häufig indirekt

Frage 99 Security

Welche Aussagen über sichere Passwörter/Authentifizierung sind korrekt?

✓ **Passwörter sollten niemals im Klartext gespeichert werden**

✓ **HTTPS hilft, Daten auf dem Transportweg zu schützen**

„123456“ ist ein gutes Passwort, weil es leicht zu merken ist

✓ **Multi-Factor Authentication (MFA) kann Sicherheit erhöhen**

Antworten:

- Passwörter sollten niemals im Klartext gespeichert werden
- HTTPS hilft, Daten auf dem Transportweg zu schützen
- Multi-Factor Authentication (MFA) kann Sicherheit erhöhen

Frage 100 Web Basics

Welche Aussagen über den Ablauf eines Webseitenaufrufs sind korrekt?

✓ **Der Browser sendet eine Anfrage (Request) und erhält eine Antwort (Response)**

✓ **Der Browser rendert die erhaltenen Ressourcen (HTML/CSS/JS) zur Darstellung**

Der Server rendert immer die UI im Browser, der Browser zeigt nur Bilder

✓ **Zusätzliche Ressourcen (CSS/JS/Bilder) werden oft separat nachgeladen**

Antworten:

- Der Browser sendet eine Anfrage (Request) und erhält eine Antwort (Response)
- Der Browser rendert die erhaltenen Ressourcen (HTML/CSS/JS) zur Darstellung
- Zusätzliche Ressourcen (CSS/JS/Bilder) werden oft separat nachgeladen